

Univerzitet u Beogradu
Matematički fakultet

Package Analyzer

Analizator zavisnosti softverskih paketa

Seminarski rad iz predmeta
Dizajn i analiza algoritama

Student: Valentina Dragić

Indeks: 078/2022

Jezik: C++20

Algoritmi: BFS, Tarjanov SCC, Kahnov topsort

Beograd, 2026.

Contents

1	Uvod	2
1.1	Motivacija	2
2	Opis problema i ulazni format	2
2.1	Registar zavisnosti	2
2.2	Lista potrebnih paketa	3
2.3	Matematički model	3
3	Arhitektura projekta	3
3.1	Struktura čvora	3
3.2	Graf kao model	4
4	Algoritmi	4
4.1	Tarjanov algoritam za jako povezane komponente	4
4.1.1	Teorijska osnova	4
4.1.2	Opis algoritma	4
4.1.3	Implementacija	5
4.1.4	Zašto Tarjan, a ne Kahn za detekciju ciklusa	5
4.2	Kahnov algoritam za topološko sortiranje	6
4.2.1	Teorijska osnova	6
4.2.2	Adaptacija za obrnutu semantiku ivica	6
4.3	BFS za tranzitivne zavisnosti	7
4.3.1	Teorijska osnova	7
5	Interaktivni interfejs	8
6	Poređenje sa pravim alatima	9
7	Zaključak	10

1. Uvod

Svaki savremeni softverski ekosistem — Python, Node.js, Rust, Java — oslanja se na *menadžere paketa* koji automatski razrešavaju zavisnosti između biblioteka. Kada korisnik zatraži instalaciju paketa `flask`, sistem mora da shvati da je potrebno instalirati i `werkzeug`, `jinja2`, `click`, i sve njihove tranzitivne zavisnosti, i to u ispravnom redosledu.

Ovaj problem se prirodno modelira kao problem na usmerenim grafovima:

- **Čvor** predstavlja jedan softverski paket
- **Usmerena ivica** $A \rightarrow B$ znači da paket A zavisi od paketa B , odnosno da B mora biti instaliran pre A

Projekat implementira konzolni analizator koji gradi takav graf iz tekstualnog registra zavisnosti i primenjuje klasične algoritme teorije grafova za analizu: Tarjanov algoritam za pronalaženje jako povezanih komponenti, Kahnov algoritam za topološko sortiranje i BFS za analizu dostižnosti.

1.1. Motivacija

Isti grafovski problemi koji se rešavaju u ovom projektu pojavljuju se u industrijskim alatima:

Problem	Ovaj projekat	Pravi alati
Kružne zavisnosti	Tarjanov SCC	pip, npm, apt
Redosled instalacije	Kahnov topsort	pip, npm, Maven
Tranzitivne zavisnosti	BFS	pip show, npm list

Table 1: Poređenje sa industrijskim alatima

2. Opis problema i ulazni format

2.1. Registar zavisnosti

Ulazni fajl `packages.txt` sadrži registar svih poznatih paketa u formatu:

```

1 # Komentar
2 flask: werkzeug, jinja2, click, itsdangerous
3 jinja2: markupsafe
4 werkzeug: markupsafe
5 requests: urllib3, certifi, charset-normalizer, idna
6 click:
```

Listing 1: Format registra zavisnosti

Svaki red definiše jedan paket i listu njegovih direktnih zavisnosti odvojenih zarezima. Paketi bez zavisnosti mogu imati praznu desnu stranu ili je izostaviti. Redovi koji počinju sa `#` i prazni redovi se ignorišu.

2.2. Lista potrebnih paketa

Fajl `needed.txt` sadrži listu paketa koje korisnik želi da instalira:

```
1 # REST API projekat
2 fastapi
3 sqlalchemy
4 pyjwt
5 celery
```

Listing 2: Format liste potrebnih paketa

Program automatski pronalazi sve tranzitivne zavisnosti navedenih paketa.

2.3. Matematički model

Neka je P skup svih paketa i $E \subseteq P \times P$ skup zavisnosti. Graf zavisnosti $G = (P, E)$ je usmeren graf gde ivica $(u, v) \in E$ znači da paket u zavisi od paketa v .

Cilj je:

1. Detektovati sve jako povezane komponente veličine ≥ 2 (kružne zavisnosti)
2. Pronaći topološko uređenje skupa traženih paketa (redosled instalacije)
3. Za dati paket p pronaći skup $\text{dep}^*(p) = \{v \in P \mid v \text{ je dostižan iz } p\}$ (sve tranzitivne zavisnosti)

3. Arhitektura projekta

Projekat je organizovan po principu jedne odgovornosti (*Single Responsibility Principle*) — svaki modul ima tačno jednu ulogu.

Modul	Odgovornost
<code>paket.hpp</code>	Struktura čvora grafa: naziv i id
<code>parser.hpp/cpp</code>	Čitanje fajlova, vraća edge list
<code>graph.hpp/cpp</code>	Graf kao jedini model podataka
<code>algoritmi.hpp/cpp</code>	Tarjan, Kahn, BFS — slobodne funkcije
<code>komande.hpp/cpp</code>	Logika svake korisničke komande
<code>petlja.hpp/cpp</code>	Interaktivna petlja
<code>main.cpp</code>	Pokretanje

Table 2: Struktura modula

3.1. Struktura čvora

Čvor grafa:

```
1 struct Paket {
2     std::string naziv;
3     int id;
4 }
```

```

5   Paket() : naziv(""), id(-1) {}
6   Paket(const std::string& naziv, int id)
7       : naziv(naziv), id(id) {}
8 };

```

Listing 3: paket.hpp

3.2. Graf kao model

Graf kao model podataka. Parser vraća sirove stringove (edge list), graf ih prima i dodeljuje identifikatore, a sav ostali kod komunicira isključivo kroz te identifikatore.

```

1 class Graf {
2 public:
3     void popuni(const std::vector<Ivica>& ivice,
4                 const std::vector<std::string>& svaPaketi);
5
6     int brojPaketa() const;
7     const Paket& paket(int id) const;
8     const std::vector<int>& susedi(int id) const;
9     int nadjiId(const std::string& naziv)
10        const;
11 };

```

Listing 4: Javni interfejs klase Graf

4. Algoritmi

4.1. Tarjanov algoritam za jako povezane komponente

4.1.1. Teorijska osnova

Jako povezana komponenta (SCC) usmerenog grafa $G = (V, E)$ je maksimalan skup čvorova $C \subseteq V$ takav da za svaka dva čvora $u, v \in C$ postoji put od u do v i put od v do u .

U kontekstu zavisnosti paketa, SCC veličine ≥ 2 predstavlja *kružnu zavisnost* tj. paketi u toj komponenti čekaju jedan na drugog i nije ih moguće instalirati.

4.1.2. Opis algoritma

Tarjanov algoritam pronalazi sve SCC-ove u jednom DFS prolasku složenosti $O(V + E)$. Svaki čvor dobija dva broja:

- $\text{preorder}[v]$ — redni broj prvog poseta čvora v
- $\text{lowlink}[v]$ — najmanji preorder dostižan iz podstabla čvora v (uključujući v sam)

Čvor v je koren svoje SCC ako i samo ako važi:

$$\text{lowlink}[v] = \text{preorder}[v]$$

Kada se otkrije koren, svi čvorovi iznad njega na steku čine jednu SCC.

4.1.3. Implementacija

```
1  static void tarjanDFS(int cvor,
2                        const Graf& graf,
3                        int& brojac,
4                        std::vector<int>& preorder,
5                        std::vector<int>& lowlink,
6                        std::vector<bool>& onStack,
7                        std::stack<int>& stack,
8                        std::vector<std::vector<int>>&
9                        components) {
10
11     preorder[cvor] = brojac;
12     lowlink[cvor]  = brojac;
13     brojac++;
14
15     onStack[cvor] = true;
16     stack.push(cvor);
17
18     for (int sused : graf.susedi(cvor)) {
19         if (preorder[sused] == -1) {
20             tarjanDFS(sused, graf, brojac, preorder, lowlink,
21                       onStack, stack, components);
22             lowlink[cvor] = std::min(lowlink[cvor],
23                                     lowlink[sused]);
24         } else if (onStack[sused]) {
25             lowlink[cvor] = std::min(lowlink[cvor],
26                                     preorder[sused]);
27         }
28     }
29
30     if (lowlink[cvor] == preorder[cvor]) {
31         std::vector<int> komponenta;
32         while (true) {
33             int vrh = stack.top(); stack.pop();
34             onStack[vrh] = false;
35             komponenta.push_back(vrh);
36             if (vrh == cvor) break;
37         }
38         components.push_back(komponenta);
39     }
40 }
```

Listing 5: Tarjanov DFS

4.1.4. Zašto Tarjan, a ne Kahn za detekciju ciklusa

Kahnov algoritam detektuje postojanje ciklusa kao sporedni efekat topološkog sortiranja – ako na kraju nisu obrađeni svi čvorovi, postoji ciklus. Međutim, Kahn ne daje informaciju o tome koji paketi čine ciklus.

Tarjanov algoritam daje lokaciju ciklusa – vraća tačno koje pakete treba popraviti.

U registru sa stotinama paketa, razlika između “ima ciklus” i “paketA $\leftrightarrow$ paketB $\leftrightarrow$ paketC čine ciklus” je ključna za dijagnostiku.

Isti obrazac koriste industrijski alati:

- **jscycles** – alat za detekciju kružnih zavisnosti u JavaScript/TypeScript projektima, pisan u Rustu. Eksplicitno koristi Tarjanov SCC algoritam: gradi graf zavisnosti fajlova, pronalazi sve SCC-ove, i prijavljuje one veličine ≥ 2 kao cikluse sa tačnim imenima fajlova.
- **circular-dependency-plugin** za webpack – plugin koji detektuje kružne zavisnosti tokom bundlovanja JavaScript modula. Jedan od kontributora je zamenio prethodni algoritam Tarjanovim SCC algoritmom zbog performansi, navodeći da je prethodni pristup kvadratičan dok je Tarjan linearan $O(V + E)$.
- **jadecy** – Java alat za analizu zavisnosti između klasa i paketa. Koristi Tarjanov algoritam za pronalaženje SCC-ova i prijavljuje kružne zavisnosti sa tačnom listom klasa.

Naša implementacija prati isti obrazac: Tarjan se pokreće nad celim registrom pri pokretanju programa, pre nego što korisnik postavi bilo kakav upit, jer je kružna zavisnost greška u registru koja se mora prijaviti bez obzira na to šta korisnik traži.

4.2. Kahnov algoritam za topološko sortiranje

4.2.1. Teorijska osnova

Topološko uređenje usmerenog acikličnog grafa (DAG) $G = (V, E)$ je linearno uređenje čvorova $v_1, v_2, \dots, v_n$ takvo da za svaku ivicu $(v_i, v_j) \in E$ važi $i < j$.

U kontekstu instalacije paketa: ako paket u zavisi od paketa v , tada v mora doći pre u u redosledu instalacije.

4.2.2. Adaptacija za obrnutu semantiku ivica

Naš graf ima ivicu $u \rightarrow v$ koja znači “ u zavisi od v ”, dakle v mora biti instaliran *pre* u . Direktna primena Kahnovog algoritma dala bi obrnut redosled, pa je neophodno raditi na invertovanom grafu.

```

1 static std::vector<int> kahnSort(const Graf& graf,
2                                 const std::vector<int>&
3                                 podskup) {
4     std::vector<bool> ukljucen(graf.brojPaketa(), false);
5     for (int id : podskup) ukljucen[id] = true;
6
7     // preostalo[u] = broj zavisnosti koje cvor u jos nije
8     // "resio"
9     std::vector<int> preostalo(graf.brojPaketa(), 0);
10    std::vector<std::vector<int>> invertovan(graf.brojPaketa());
11
12    for (int u = 0; u < graf.brojPaketa(); u++) {
13        if (!ukljucen[u]) continue;
14        for (int v : graf.susedi(u)) {
15            if (!ukljucen[v]) continue;

```

```

14         preostalo[u]++;
15         invertovan[v].push_back(u);
16     }
17 }
18
19 // paketi bez zavisnosti mogu biti instalirani odmah
20 std::queue<int> red;
21 for (int i = 0; i < graf.brojPaketa(); i++)
22     if (uključen[i] && preostalo[i] == 0) red.push(i);
23
24 std::vector<int> redosled;
25 while (!red.empty()) {
26     int cvor = red.front(); red.pop();
27     redosled.push_back(cvor);
28     for (int zavisnik : invertovan[cvor])
29         if (--preostalo[zavisnik] == 0) red.push(zavisnik);
30 }
31
32 return redosled; // prazan ako postoji ciklus
33 }

```

Listing 6: Kahnov algoritam na podskupu čvorova

Ako na kraju veličina rezultata nije jednaka veličini podskupa, postoji ciklus — Kahn to detektuje kao sporedni efekat, ali bez informacije o lokaciji ciklusa (zbog čega i koristimo Tarjana).

4.3. BFS za tranzitivne zavisnosti

4.3.1. Teorijska osnova

Za dati paket p , skup tranzitivnih zavisnosti je:

$$\text{dep}^*(p) = \{v \in V \mid \exists \text{ put od } p \text{ do } v \text{ u } G\} \setminus \{p\}$$

BFS iz čvora p po ivicama grafa zavisnosti daje tačno ovaj skup. Složenost je $O(V + E)$.

```

1 std::vector<int> tranzitivneZavisnosti(const Graf& graf, int
  startId) {
2     std::vector<bool> posecen(graf.brojPaketa(), false);
3     std::queue<int> red;
4
5     posecen[startId] = true;
6     red.push(startId);
7
8     std::vector<int> rezultat;
9
10    while (!red.empty()) {
11        int cvor = red.front(); red.pop();
12        for (int sused : graf.susedi(cvor)) {
13            if (!posecen[sused]) {
14                posecen[sused] = true;

```



```

15         red.push(sused);
16         rezultat.push_back(sused);
17     }
18 }
19 }
20
21 return rezultat;
22 }

```

Listing 7: BFS za tranzitivne zavisnosti

BFS se koristi na dva mesta: u komandi `deps` za direktan prikaz zavisnosti, i interno u `napraviPodskup` za određivanje skupa paketa koji će biti analizirani pri `install` i `needed` komandama.

5. Interaktivni interfejs

Program nudi interaktivni konzolni interfejs, graf se učitava jednom, a korisnik postavlja upite bez ponovnog pokretanja:

Komanda	Opis	Algoritam
<code>install <package></code>	Redosled instalacije	Kahn
<code>deps <package></code>	Tranzitivne zavisnosti	BFS
<code>info <package></code>	Direktne zavisnosti	Pristup listi susedstva
<code>cycles</code>	Kružne zavisnosti u registru	Tarjan SCC
<code>isolated</code>	Paketi bez veza	Linearna provera
<code>needed <file></code>	Instalacija iz fajla	BFS + Kahn
<code>load <registry></code>	Apdejt grafa u slučaju promena u registru	

Table 3: Dostupne komande

Primer sesije:

```

1 $ ./dep-analyzer
2 =====
3         PACKAGE DEPENDENCY ANALYZER
4 =====
5
6 Loaded: 138 packages, 162 dependencies.
7   No circular dependencies found. Graph is acyclic (DAG).
8
9
10 Options:
11   install <package>      show installation order for package and
12     dependencies
13   deps    <package>      list transitive dependencies
14   info    <package>      show direct dependencies
15   load    <registry>      load registry again in case of change
16   cycles                      detect circular dependencies

```

```
16  isolated                show isolated packages
17  needed <file>           load needed file and show installation
    order
18  help                    show available commands
19  quit                    exit program
20
21 > install flask
22
23 Installation order for package 'flask':
24 1. click
25 2. itsdangerous
26 3. markupsafe
27 4. jinja2
28 5. werkzeug
29 6. flask
30
31 > deps requests
32
33 Transitive dependencies for package 'requests' (4):
34 - urllib3
35 - certifi
36 - charset-normalizer
37 - idna
38
39 > install unknown
40 Error: package 'unknown' not in registry.
41 > quit
```

Listing 8: Primer interaktivne sesije

6. Poređenje sa pravim alatima

Projekat implementira algoritmsku osnovu svakog paket menadžera. Razlika između ovog projekta i alata poput pip-a ili npm-a nije u algoritmima, isti Tarjan, Kahn i BFS se koriste. Razlika je u perifernim delovima koji nisu tema kursa (teorije grafova):

- **Mrežni pristup** – pravi alati preuzimaju pakete sa centralnog repozitorijuma (PyPI, npm registry, apt sources)
- **Verzionisanje** – pravi alati podržavaju semver ograničenja poput `werkzeug >= 2.0, < 3.0`
- **Razrešavanje konflikata** – moderni pip koristi SAT solver (backtracking) za pronalaženje kompatibilnog skupa verzija; ovo je NP-kompletna problem u opštem slučaju
- **Izolacija okruženja** – `virtualenv`, `venv`, `node_modules` po projektu

Ove razlike su namerno izostavljene – fokus projekta je na grafovskim algoritmima, ne na implementaciji kompletnog paket menadžera.

7. Zaključak

Projekat demonstrira da se problem upravljanja zavisnostima softverskih paketa može svesti na klasične probleme teorije grafova: detekciju jako povezanih komponenti, topološko sortiranje i analizu dostižnosti.

Implementirani algoritmi – Tarjanov SCC, Kahnov topsort i BFS – imaju složenost $O(V + E)$ i direktno odgovaraju algoritmima koji se koriste u industrijskim paket-menadžerima. Arhitektura projekta je organizovana po principu jedne odgovornosti, bez globalnog stanja i sa jasnom podelom između strukture podataka, algoritama i korisničkog interfejsa.

References

- [1] Vesna Marinković, Filip Marić, *Konstrukcija i analiza algoritama*, Matematički fakultet, Univerzitet u Beogradu. Dostupno na: <https://poincare.matf.bg.ac.rs/~filip.maric/kiaa/kiaa.pdf>
- [2] EnderHub, *jscycles – Fast circular dependency detection for JS/TS*, GitHub. <https://github.com/EnderHub/jscycles>
- [3] hedgepigdaniel, *Replace cycle detection algorithm with Tarjan's SCC algorithm*, Pull Request #49, circular-dependency-plugin, GitHub, 2019. <https://github.com/aackerman/circular-dependency-plugin/pull/49>